Architecture Overview

High-level overview of the Solana KYC Compliance SDK system design and data flow.

Table of Contents

1. System Overview
2. Architecture Diagram
3. Core Components
4. Data Flow
5. Security Model
6. Scalability Considerations

System Overview

The Solana KYC Compliance SDK is a decentralized compliance layer that enforces KYC/AML requirements directly at the token level using Solana's Token Extensions (Token-2022). The system consists of three main layers:

1. **On-Chain Layer** - Rust programs running on Solana
2. **SDK Layer** - TypeScript client for integration
3. **Intelligence Layer** - Off-chain risk assessment and monitoring

---

Architecture Diagram

```
graph TB
    subgraph "Client Applications"
        A[Web App]
        B[Mobile App]
        C[Backend Service]
    end

    subgraph "SDK Layer"
        D[ComplianceSDK]
        E[TransferHookService]
        F[RiskEngine]
    end

    subgraph "Solana Blockchain"
        G[Token-2022 Program]
        H[KYC Compliance Program]
        I[Compliance Registry]
        J[Circuit Breaker]
    end

    subgraph "Off-Chain Services"
        K[Oracle Fetcher]
        L[Risk Scoring Engine]
        M[External Intelligence APIs]
    end

    subgraph "Admin Dashboard"
        N[Risk Dashboard]
        O[Registry Management]
    end

    A --> D
    B --> D
    C --> D
    D --> E
    D --> F
    E --> G
    G --> H
    H --> I
    H --> J
```

```
    K --> M
    K --> L
    L --> I
    N --> O
    O --> H
```

Core Components

1. On-Chain Programs

KYC Compliance Program (`kyc-compliance`)

The main program that enforces KYC checks on token transfers using the Transfer Hook interface.

Key Features:
- Transfer hook implementation for Token-2022
- PDA-based KYC status verification
- Real-time compliance checks
- Event emission for indexing

Program ID: `KYCCo7vM2uLkGzqH6XqKJp1TJK5JjK9jW8vY9xQz1P2`

Compliance Registry (`compliance_registry`)

Stores and manages the allowlist/blocklist of verified wallets.

Data Structures:
- `KYCStatus` - Individual wallet verification status
- `ComplianceRegistry` - Global registry state
- `RiskScore` - Wallet risk assessment

2. TypeScript SDK

The client-side SDK provides a unified interface for interacting with the on-chain programs.

Modules:

- `ComplianceSDK` - Main interface for KYC operations
- `TransferHookService` - Token Extension management
- `RiskEngine` - Risk assessment client
- `types` - TypeScript type definitions
- `errors` - Error handling utilities

3. Risk Engine

Off-chain service that calculates and monitors wallet risk scores.

Components:

- **Scoring Engine** - Calculates risk scores based on multiple factors
- **Oracle Fetcher** - Fetches external intelligence data
- **Alert Service** - Monitors and alerts on high-risk activity

4. Admin Dashboard

React-based UI for compliance officers to manage the system.

Features:
- Real-time risk monitoring
- Wallet management (freeze/unfreeze)
- Transaction history
- Alert management

Data Flow

Registration Flow

```
sequenceDiagram
    participant User
    participant SDK
    participant Blockchain
    participant Registry

    User->>SDK: registerUser(kycData)
    SDK->>Blockchain: Submit transaction
    Blockchain->>Registry: Create KYC account
    Registry-->>Blockchain: Confirm registration
    Blockchain-->>SDK: Return signature
    SDK-->>User: Registration complete
```

Transfer Flow

```
sequenceDiagram
    participant Sender
    participant TokenProgram
    participant TransferHook
    participant Registry
    participant Recipient

    Sender->>TokenProgram: Initiate transfer
    TokenProgram->>TransferHook: Execute transfer hook
    TransferHook->>Registry: Check sender status
    TransferHook->>Registry: Check recipient status
    alt Both verified
        Registry-->>TransferHook: Approved
        TransferHook-->>TokenProgram: Proceed
        TokenProgram->>Recipient: Transfer tokens
    else Not verified
        Registry-->>TransferHook: Rejected
        TransferHook-->>TokenProgram: Revert transaction
    end
```

Risk Monitoring Flow

```
sequenceDiagram
    participant Oracle
    participant ExternalAPI
    participant RiskEngine
    participant Blockchain

    Oracle->>ExternalAPI: Fetch intelligence
    ExternalAPI-->>Oracle: Sanctions data
    Oracle->>RiskEngine: Process data
    RiskEngine->>RiskEngine: Calculate scores
    RiskEngine->>Blockchain: Update registry
    Blockchain-->>RiskEngine: Confirmation
```

Security Model

1. Authority Management

- **Multi-sig Support** - Critical operations require multiple signatures
- **Role-based Access** - Different permission levels for admin operations
- **Time-locks** - Delay on sensitive operations

2. Circuit Breaker

Emergency stop mechanism that halts all transfers in case of security incidents.

Triggers:
- Admin manual activation
- Abnormal transaction volume
- Detected security breach

3. ZK Proofs for Privacy

Privacy-preserving verification using zero-knowledge proofs.

Benefits:
- Verify KYC without revealing identity
- Prevent double-spending of credentials
- Maintain user privacy

4. Rate Limiting

- Velocity limits on transfers
- Rate limiting on admin operations
- DDoS protection

Smart Contract Architecture

Account Structure

```
┌──────────────────────────────────────────┐
│           KYC Status PDA                 │
├──────────────────────────────────────────┤
│ wallet: Pubkey          (32 bytes)       │
│ status: KycStatus       (1 byte)         │
│ kycLevel: KycLevel      (1 byte)         │
│ countryCode: String     (variable)       │
│ expiryTimestamp: i64   (8 bytes)        │
│ registeredAt: i64       (8 bytes)       │
│ lastUpdated: i64         (8 bytes)       │
└──────────────────────────────────────────┘


┌──────────────────────────────────────────┐
│       Compliance Registry PDA            │
├──────────────────────────────────────────┤
│ authority: Pubkey     (32 bytes)         │
│ totalRegistered: u64  (8 bytes)          │
│ paused: bool           (1 byte)          │
│ circuitBreaker: bool   (1 byte)          │
│ maxTransfer: u64       (8 bytes)          │
│ velocityLimit: u64     (8 bytes)          │
│ velocityWindow: u32    (4 bytes)         │
│ bump: u8               (1 byte)           │
└──────────────────────────────────────────┘
```

Instruction Set

Instruction	Description	Authority
`register_user`	Register new KYC	User
`update_status`	Update KYC status	Admin
`freeze_wallet`	Freeze a wallet	Admin
`unfreeze_wallet`	Unfreeze a wallet	Admin
`activate_circuit_breaker`	Halt all transfers	Multi-sig

Instruction	Description	Authority
`deactivate_circuit_breaker`	Resume transfers	Multi-sig
`verify_zkp`	Verify ZK proof	User

Scalability Considerations

On-Chain Optimization

- **PDA Usage** - Efficient account derivation
- **Cross-program Invocation** - Minimize compute usage
- **Event-based Indexing** - Reduce on-chain reads

Off-Chain Scaling

- **Caching** - Cache frequently accessed data
- **Batch Processing** - Process multiple operations together
- **Load Balancing** - Distribute API load

Network Effects

- **Transaction Batching** - Group similar operations
- **Priority Fees** - Ensure critical transactions process
- **Failover** - Multiple RPC endpoints

Upgrade Path

The system is designed for future upgrades:

1. **Program Upgrades** - Using upgradeable BPF
2. **SDK Versions** - Backward-compatible updates
3. **New Features** - Modular architecture for easy addition

Related Documentation

- API Reference
- Integration Guide
- Security Policy
- GitHub Repository